MemoryManager

Sistema de Gestión de Memoria con Garbage Collection Multihilo en Java

Java 8+ | Concurrencia | Thread Safety | Garbage Collection | ThreadPoolExecutor | AtomicInteger | ConcurrentHashMap

Documentación técnica completa del sistema de gestión de memoria personalizado que simula garbage collection en un entorno multihilo, con seguimiento de referencias y eliminación automática de objetos sin referencias activas.

Fecha: Abril 2026 | **Lenguaje:** Java 8+ | **Archivo:** MemoryManager.java

Tabla de Contenidos

1. Descripción General y Contexto del Problema
2. Arquitectura del Sistema y Diagrama de Flujo
3. Clase `MemoryObject` — Análisis detallado
4. Clase `MemoryManager` — Todos sus métodos
5. Clase `Solution` y método `main`
6. Thread Safety — Análisis profundo
7. Casos de Uso, Ejemplos y Trazas de Ejecución
8. Restricciones y Consideraciones de Diseño
9. Código Fuente Completo

1. Descripción General y Contexto del Problema

En Java, la JVM tiene su propio garbage collector. Este proyecto **simula cómo funciona ese proceso desde cero**, añadiendo la complejidad de un entorno **multihilo**: varios hilos pueden añadir o quitar referencias al mismo objeto al mismo tiempo.

> **Objetivo principal:**
> Simular la gestión de memoria a bajo nivel — creación de objetos, conteo de referencias, liberación automática por garbage collection — con correctitud garantizada bajo ejecución concurrente con hasta 5 hilos simultáneos.

¿Qué es el conteo de referencias?

Es una técnica clásica de gestión de memoria. Cada objeto lleva un contador que registra cuántas partes del programa lo "usan". Cuando el contador llega a cero, el objeto no tiene dueño y puede eliminarse de forma segura.

Clase	Responsabilidad
`MemoryObject`	Representa un objeto en memoria con ID y contador de referencias
`MemoryManager`	Gestiona el pool de memoria con operaciones thread-safe
`Solution`	Lee comandos, orquesta el ThreadPool y escribe resultados

2. Arquitectura del Sistema

```
stdin (n comandos)
        |
+-------v----------------------------------------+
|              Solution.main()                    |
|  Lee comandos  →  crea ThreadPoolExecutor(5)    |
+-------+----------------------------------------+
        | submit() x n tareas
        v
   [Hilo-1]  [Hilo-2]  [Hilo-3]  [Hilo-4]  [Hilo-5]
        +----+----+----+----+----+
             |
             v  executeCommand(command, memoryManager)
   +----------------------------------------+
   |          MemoryManager                 |
   |  ConcurrentHashMap<String,MemoryObject> |
   |  addReference  /  removeReference      |
   |  garbageCollect  /  getMemoryPool      |
   +----------------------------------------+
             |
             |  future.get() x n  (esperar todos)
             v
   getMemoryPool() → filtra >=0, ordena A→Z
             |
             v
   OUTPUT_PATH (archivo de salida)
```

Ciclo de vida de un objeto

```
addReference("obj1")    →  obj1 creado, count: 0 → 1
addReference("obj1")    →  count: 1 → 2
removeReference("obj1") →  count: 2 → 1
removeReference("obj1") →  count: 1 → 0
garbageCollect()        →  count=0 → obj1 ELIMINADO del pool
```

```
removeReference("x")    →  x creado, count: 0 → -1  (orden invertido por concurrencia)
addReference("x")       →  count: -1 → 0             (se "normaliza")
getMemoryPool()         →  x incluido (count=0 >= 0 ✓)
```

3. Clase MemoryObject

Representa la unidad básica del pool. Cada instancia almacena el identificador único del objeto y cuántas referencias activas tiene en el sistema.

```
class MemoryObject {
    String objectId;
    AtomicInteger referenceCount;

    MemoryObject(String objectId) {
        this.objectId = objectId;
        this.referenceCount = new AtomicInteger(0);  // Inicia en 0, no en 1
    }

    @Override
    public String toString() {
        return "MemoryObject{id='" + objectId + "', referenceCount=" +
referenceCount.get() + "}";
    }
}
```

¿POR QUÉ ATOMICINTEGER Y NO INT?

Con un `int` normal, dos hilos que incrementen simultáneamente producen una *race condition*:

```
Hilo A lee el valor: 5
Hilo B lee el valor: 5    ← B leyó ANTES de que A escribiera
Hilo A escribe:      6
Hilo B escribe:      6    ← Debería ser 7. Un incremento se perdió.
```

`AtomicInteger.incrementAndGet()` usa instrucciones CAS (Compare-And-Swap) a nivel de CPU. La operación es **atómica e indivisible** — nunca habrá dos lecturas simultáneas del mismo valor base.

> **¿Por qué el contador inicia en 0?**
>
> En un entorno multihilo los comandos llegan en orden no determinista. Un removeReference puede ejecutarse antes que el addReference correspondiente. Iniciar en 0 permite que el sistema absorba contadores negativos temporales sin corromper el estado final.

TOSTRING() Y EL FORMATO DE SALIDA

El método produce exactamente el formato requerido por la especificación:

```
MemoryObject{id='obj1', referenceCount=2}
```

4. Clase MemoryManager

Estructura de datos: ConcurrentHashMap

```
private ConcurrentHashMap<String, MemoryObject> memoryPool = new ConcurrentHashMap<>();
```

Estructura	Thread-safe	Bloquea todo el mapa	Rendimiento concurrente
`HashMap`	No	—	Puede corromperse
`synchronizedMap`	Sí	Sí (un lock global)	Regular
`ConcurrentHashMap`	Sí	No (lock por segmento)	Excelente

---

createObject

```
public void createObject(String objectId) {
    memoryPool.putIfAbsent(objectId, new MemoryObject(objectId));
}
```

`putIfAbsent` es atómico: crea el objeto SOLO si no existe. El equivalente con `if/put` separados no sería thread-safe — entre el `containsKey` y el `put`, otro hilo podría insertar el mismo objeto.

---

addReference y removeReference

```
public void addReference(String objectId) {
    memoryPool.computeIfAbsent(objectId, MemoryObject::new)
              .referenceCount.incrementAndGet();
}

public void removeReference(String objectId) {
    memoryPool.computeIfAbsent(objectId, MemoryObject::new)
              .referenceCount.decrementAndGet();
}
```

`computeIfAbsent(key, fn)` realiza dos acciones en una sola operación atómica:

1. Si la clave no existe → crea el objeto con `MemoryObject::new`

2. Devuelve el objeto (nuevo o existente) para encadenar la operación

Luego se incrementa/decrementa el `AtomicInteger`. `removeReference` puede generar contadores negativos — esto es intencional y correcto en concurrencia.

---

garbageCollect

```
public synchronized void garbageCollect() {
    memoryPool.entrySet().removeIf(e -> e.getValue().referenceCount.get() <= 0);
}
```

¿Por qué synchronized si ya usamos ConcurrentHashMap?

ConcurrentHashMap protege operaciones individuales (get, put). Pero removeIf es una iteración + múltiples eliminaciones. Si dos hilos ejecutan GC en paralelo pueden interferir. synchronized garantiza que solo un GC corre a la vez.

Valor del contador	Acción del GC
>= 1	Objeto conservado (tiene referencias activas)
0	ELIMINADO (sin referencias)
< 0	ELIMINADO (estado temporal inválido)

---

getMemoryPool

```
public List<MemoryObject> getMemoryPool() {
    if (memoryPool.size() > 3) {    // GC automático si pool grande
        garbageCollect();
    }
    return memoryPool.values().stream()
            .filter(obj -> obj.referenceCount.get() >= 0)      // excluye negativos
            .sorted(Comparator.comparing(obj -> obj.objectId))  // orden alfabético
            .collect(Collectors.toList());                       // copia inmutable
}
```

Paso	Qué hace	Por qué
GC automático	Limpia si hay >3 objetos en el pool	Evita acumulación de basura en pools grandes
filter >= 0	Excluye objetos con contador negativo	Son estados temporales inválidos por concurrencia
sorted	Ordena alfabéticamente por objectId	La especificación lo exige para salida determinista

collect	Crea nueva lista independiente	Aísla al llamador del pool interno mutable

Diferencia clave entre GC y filtro:

garbageCollect() elimina permanentemente del mapa los objetos con count <= 0. El filtro en getMemoryPool() solo excluye los negativos de la vista retornada, dejándolos en el mapa porque aún podrían recibir referencias concurrentes.

5. Clase Solution y método main

executeCommand — Despacho de operaciones

```
public static void executeCommand(String command, MemoryManager memoryManager) {
    String[] commandParts = command.split(" ", 3);
    String action = commandParts[0];
    if (action.equals("addReference")) {
        memoryManager.addReference(commandParts[1]);
    } else if (action.equals("removeReference")) {
        memoryManager.removeReference(commandParts[1]);
    } else if (action.equals("garbageCollect")) {
        memoryManager.garbageCollect();
    }
}
```

`split(" ", 3)` divide en máximo 3 partes, por si el objectId contuviera espacios. Ejemplo: `"addReference obj1"` → `["addReference", "obj1"]`

main — El orquestador concurrente

```
ThreadPoolExecutor threadPoolExecutor =
    (ThreadPoolExecutor) Executors.newFixedThreadPool(5);
```

Crea un pool con **exactamente 5 hilos**. Con 100 comandos, los primeros 5 arrancan de inmediato; el resto espera en la cola interna hasta que un hilo quede libre.

```
Future<Void> future = threadPoolExecutor.submit(() -> {
    executeCommand(command, memoryManager);
    return null;
});
```

`submit()` devuelve un `Future<Void>` — una promesa de ejecución. La tarea puede estar corriendo o todavía en cola.

```
for (Future<Void> future : futures) future.get();
threadPoolExecutor.shutdown();
```

`future.get()` **bloquea el hilo principal** hasta que esa tarea termine. Garantiza que `getMemoryPool()` solo se llame cuando todos los comandos hayan sido procesados.

Paso	Descripción
1. Leer	Lee n y los n comandos desde stdin con BufferedReader
2. Lanzar	Envía cada comando al ThreadPool como Callable<Void>
3. Sincronizar	future.get() por cada tarea — espera el final de todas
4. Recolectar	getMemoryPool() → estado final ordenado y filtrado
5. Escribir	Escribe cada MemoryObject al archivo OUTPUT_PATH

6. Thread Safety — Análisis Profundo

Componente	Mecanismo	Garantía
Pool de objetos	`ConcurrentHashMap`	Inserciones/lecturas concurrentes sin corrupción
Contador de referencias	`AtomicInteger`	Incrementos/decrementos sin race conditions (CAS a nivel CPU)
Garbage collection	`synchronized`	Solo un GC activo a la vez, sin interferencia
Creación de objetos	`computeIfAbsent`	Creación atómica — imposible crear duplicados
Lectura final	`stream().collect()`	Snapshot inmutable, ordenado y filtrado

Escenario de carrera tolerable

```
T=0ms  Hilo-1: removeReference("obj1") → obj1 creado con count=0, decrementado a -1
T=1ms  Hilo-2: addReference("obj1")    → obj1 ya existe, count: -1 → 0
T=2ms  Hilo-3: addReference("obj1")    → count: 0 → 1
T=3ms  Hilo-4: addReference("obj1")    → count: 1 → 2

Resultado: obj1 con referenceCount=2  (idéntico al orden "normal")
```

¿Por qué no se usa synchronized en addReference/removeReference?

Porque `synchronized` en esos métodos crearía un cuello de botella severo: solo un hilo podría incrementar/decrementar a la vez, eliminando el beneficio real del ThreadPool. `AtomicInteger` ofrece la misma correctitud con mayor rendimiento usando instrucciones atómicas de hardware (sin suspender hilos).

> **Principio de diseño — mínima granularidad de bloqueo:**
> CAS (sin lock) para contadores, lock por segmento para el mapa, synchronized solo para GC que requiere iteración + eliminación múltiple.

7. Casos de Uso, Ejemplos y Trazas

Ejemplo 1 — Sample Case 0 (5 comandos)

Comando	Estado del pool tras la operación
addReference obj1	obj1: 1
addReference obj2	obj1:1, obj2:1
addReference obj1	obj1:2, obj2:1
addReference obj3	obj1:2, obj2:1, obj3:1
removeReference obj2	obj1:2, obj2:0, obj3:1

Salida (3 objetos = GC automático no se dispara; filtro >=0 incluye obj2 con count=0):

```
MemoryObject{id='obj1', referenceCount=2}
MemoryObject{id='obj2', referenceCount=0}
MemoryObject{id='obj3', referenceCount=1}
```

Ejemplo 2 — Sample Case 1 (2 comandos, orden concurrente invertido)

Comando	Estado del pool
removeReference obj	obj creado, count: 0 → -1
addReference obj1	obj1 creado, count: 0 → 1

Salida (obj tiene count=-1, filtrado por < 0; obj1 tiene count=1, incluido):

```
MemoryObject{id='obj1', referenceCount=1}
```

Ejemplo 3 — GC automático con más de 3 objetos

```
addReference A    → A:1
addReference B    → B:1
```

```
addReference C    → C:1
addReference D    → D:1
removeReference B → B:0
removeReference C → C:0
getMemoryPool()   → pool tiene 4 objetos → GC disparado automáticamente
                  → B (count=0) ELIMINADO
                  → C (count=0) ELIMINADO
                  → Retorna: [A:1, D:1]
```

8. Restricciones y Consideraciones de Diseño

Parámetro	Valor
Longitud de objectId	0 – 100 caracteres alfanuméricos
Número de comandos (n)	1 – 100
Comandos válidos	addReference, removeReference, garbageCollect
Hilos del ThreadPool	5 fijos
GC automático	Se activa en getMemoryPool si pool tiene >3 objetos
Filtro de salida	Solo objetos con count >= 0
Ordenamiento	Alfabético por objectId (A → Z)

Decisiones de diseño clave

Decisión	Alternativa descartada	Razón
AtomicInteger para contadores	synchronized int	Sin bloqueo de método; mayor throughput concurrente
ConcurrentHashMap	HashMap + synchronized	Lock granular por segmento; lectura sin bloqueo
Negativos permitidos en removeReference	Ignorar si count ya es 0	Enmascararía errores reales de lógica
computeIfAbsent en add/remove	createObject previo manual	Atómica; sin duplicados posibles
synchronized solo en GC	synchronized en todo MemoryManager	Mínima contención; máximo paralelismo

9. Código Fuente Completo

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.Collectors;

class MemoryObject {
    String objectId;
    AtomicInteger referenceCount;

    MemoryObject(String objectId) {
        this.objectId = objectId;
        this.referenceCount = new AtomicInteger(0);
    }

    @Override
    public String toString() {
        return "MemoryObject{id='" + objectId + "', referenceCount=" +
referenceCount.get() + "}";
    }
}

class MemoryManager {
    private ConcurrentHashMap<String, MemoryObject> memoryPool = new ConcurrentHashMap<>
();

    public void createObject(String objectId) {
        memoryPool.putIfAbsent(objectId, new MemoryObject(objectId));
    }

    public void addReference(String objectId) {
        memoryPool.computeIfAbsent(objectId,
MemoryObject::new).referenceCount.incrementAndGet();
    }

    public void removeReference(String objectId) {
```

```
        memoryPool.computeIfAbsent(objectId,
MemoryObject::new).referenceCount.decrementAndGet();
    }

    public synchronized void garbageCollect() {
        memoryPool.entrySet().removeIf(e -> e.getValue().referenceCount.get() <= 0);
    }

    public List<MemoryObject> getMemoryPool() {
        if (memoryPool.size() > 3) {
            garbageCollect();
        }
        return memoryPool.values().stream()
                .filter(obj -> obj.referenceCount.get() >= 0)
                .sorted(Comparator.comparing(obj -> obj.objectId))
                .collect(Collectors.toList());
    }
}

class Solution {
    public static void executeCommand(String command, MemoryManager memoryManager) {
        String[] commandParts = command.split(" ", 3);
        String action = commandParts[0];
        if (action.equals("addReference")) {
            String object = commandParts[1];
            memoryManager.addReference(object);
        } else if (action.equals("removeReference")) {
            String object = commandParts[1];
            memoryManager.removeReference(object);
        } else if (action.equals("garbageCollect")) {
            memoryManager.garbageCollect();
        }
    }

    public static void main(String[] args) throws IOException, ExecutionException,
InterruptedException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));
        int commandsCount = Integer.parseInt(bufferedReader.readLine().trim());
        List<String> commands = new ArrayList<>();
        for (int i = 0; i < commandsCount; i++) {
            String commandsItem = bufferedReader.readLine();
            commands.add(commandsItem);
        }
        ThreadPoolExecutor threadPoolExecutor = (ThreadPoolExecutor)
Executors.newFixedThreadPool(5);
        List<Future<Void>> futures = new ArrayList<>();

        MemoryManager memoryManager = new MemoryManager();
        for (String command : commands) {
            Future<Void> future = threadPoolExecutor.submit(() -> {
                executeCommand(command, memoryManager);
```

```
                return null;
            });
            futures.add(future);
        }

        // Wait for all tasks to complete
        for (Future<Void> future : futures) {
            future.get();
        }
        threadPoolExecutor.shutdown();

        // Collect the memory pool state after all commands have been processed
        List<String> result = new ArrayList<>();
        for (MemoryObject memoryObject : memoryManager.getMemoryPool()) {
            result.add(memoryObject.toString());
        }
        for (int i = 0; i < result.size(); i++) {
            bufferedWriter.write(result.get(i));
            if (i != result.size() - 1) {
                bufferedWriter.write("\n");
            }
        }
        bufferedWriter.newLine();
        bufferedReader.close();
        bufferedWriter.close();
    }
}
```